

Delivery Delay & Operations Analytics

Jan 2025 – Dec 2025 · 15,000 orders · 5 warehouses · 4 carriers · 7 product categories · 20 routes

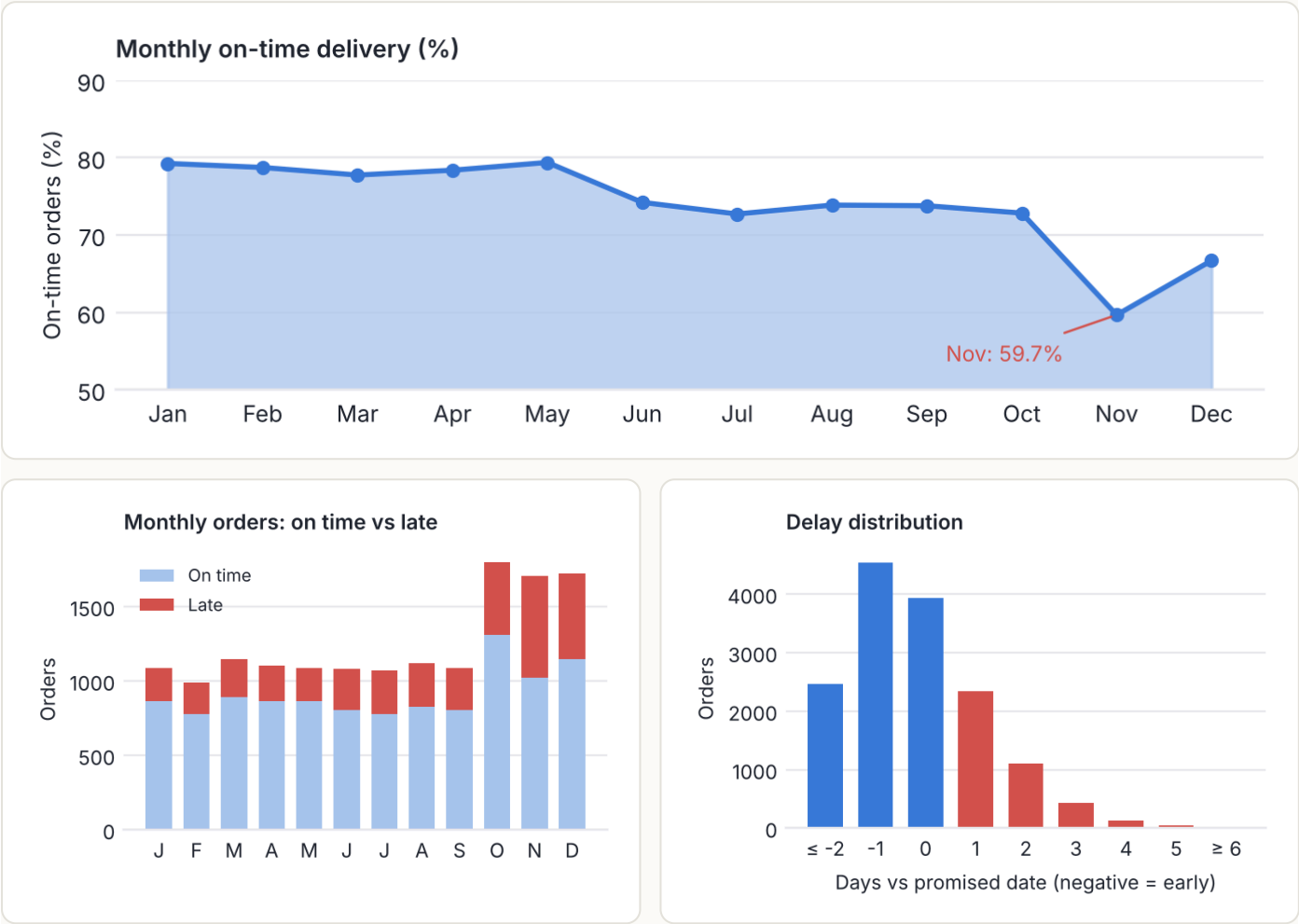
About the data: no real dataset was provided, so this analysis uses a synthetic dataset generated by generate_data.py (fixed seed). Delay patterns, including three one-off incidents, are planted deliberately so the analysis has signal to find. Figures illustrate the method, not a real business.

Orders analysed	On-time delivery	Late orders
15,000	73.0%	4,054
Avg days late (late orders)	Severely late (3+ days)	Total late-days
1.64	4.1%	6,633

Summary

- 73.0% of 15,000 orders arrived on time; 4,054 were late, adding up to 6,633 late-days. 611 orders (4.1%) were 3+ days late.
- Dispatch lag is the biggest lever. Late rate is 7.8% / 27.6% / 69.7% for same-day / 1-day / 2-day dispatch.
- SwiftLine and Remote destinations are the worst segments (37.1% and 57.5% late).
- Q4 peak hurts: on-time delivery drops to 59.7% in November; Mumbai Hub is the festive and Friday hotspot.
- 54% of late orders have no logged reason, a data gap worth fixing.
- The risk model works: ROC-AUC 0.88; the top-risk decile is late 86% of the time versus 27% on average.

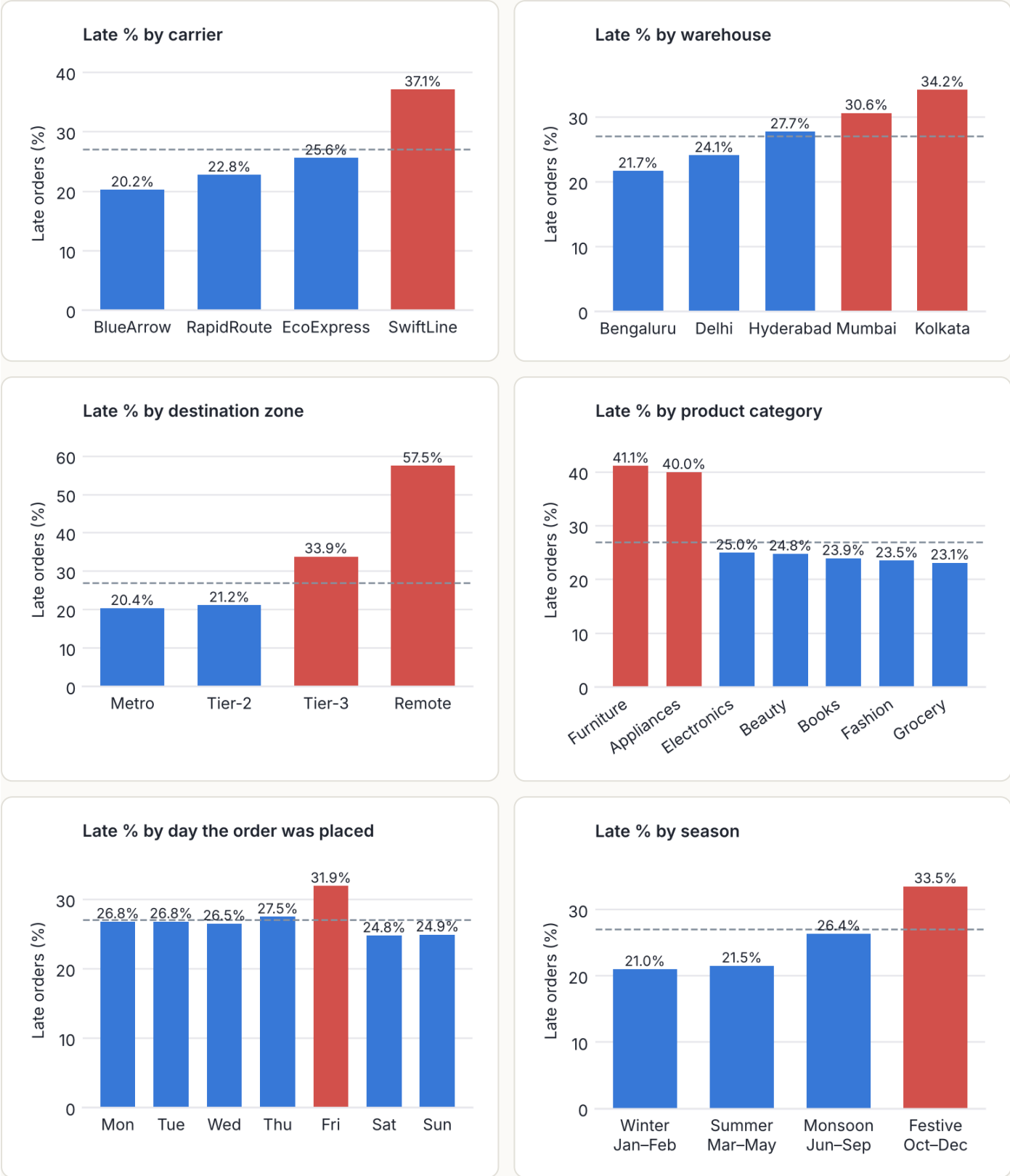
1. Delivery performance over time



On-time delivery held at roughly 79% from January to May, slipped in the monsoon months and fell to 59.7% in November as order volume surged. Most late orders are only 1–2 days late; 4.1% of all orders are 3+ days late.

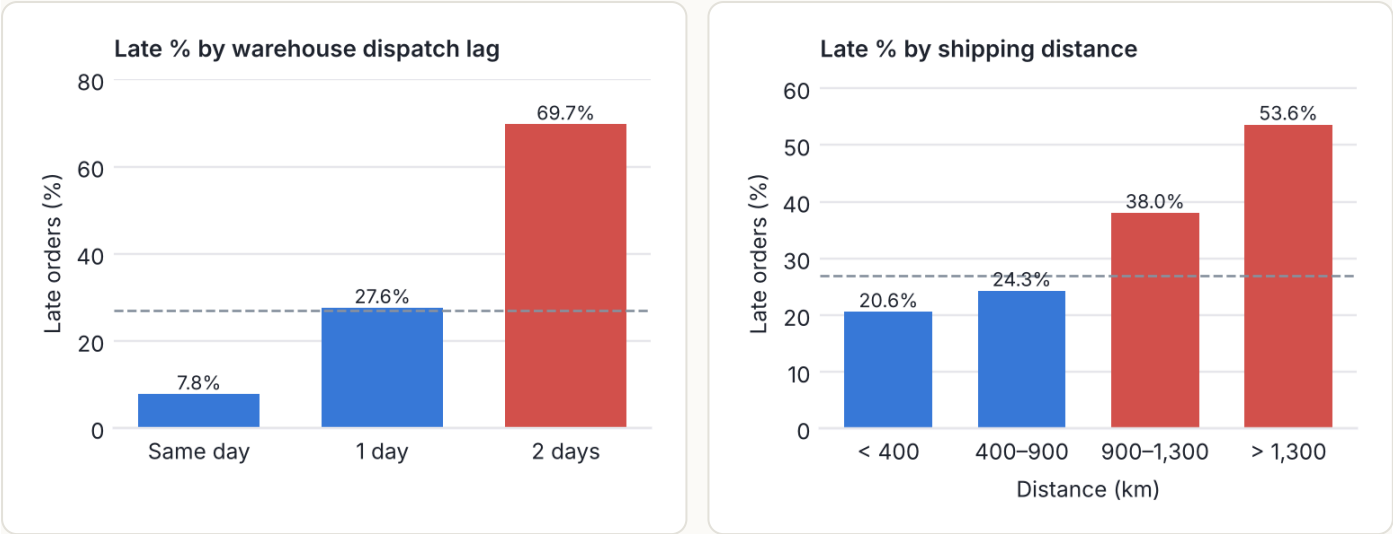
2. Where delays concentrate

Red bars are more than 10% above the overall late rate (27.0%, dashed line).

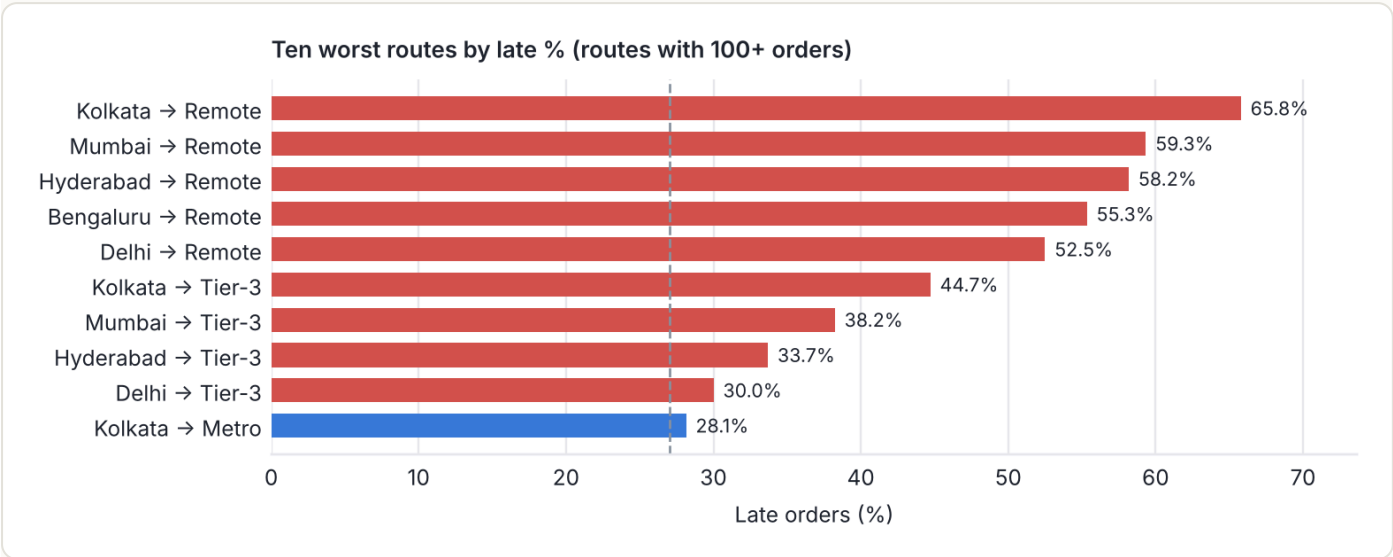


SwiftLine (37.1%) trails **BlueArrow** (20.2%) by 17 points. **Kolkata Hub** is the weakest warehouse (34.2%) and **Bengaluru Hub** the strongest (21.7%). Remote destinations (57.5%) and bulky categories (Furniture, Appliances) stand out most. Weekday differences are small overall; the Friday effect is concentrated at Mumbai Hub.

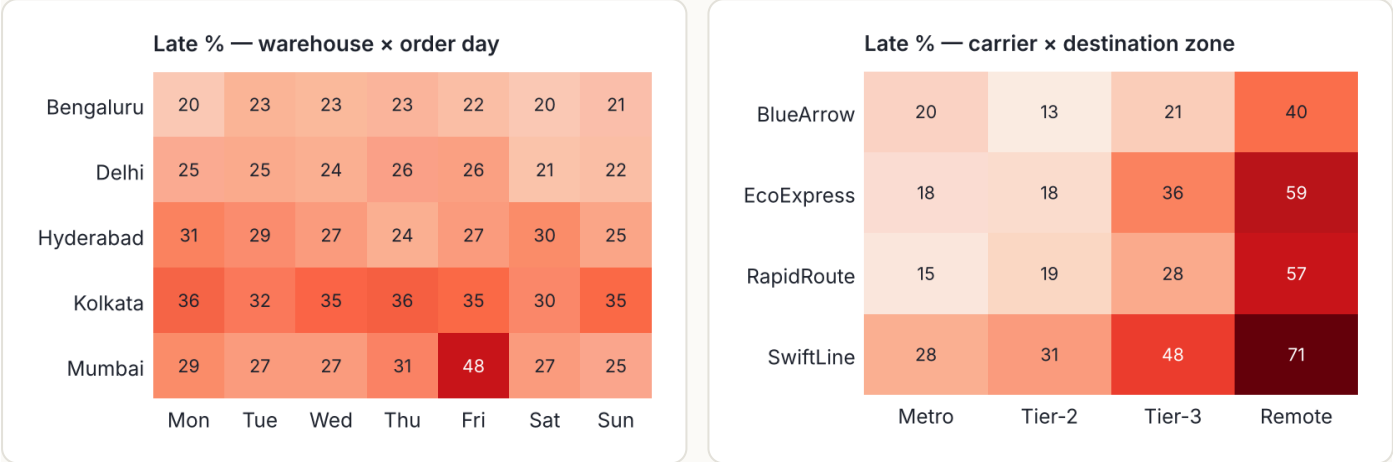
3. Dispatch lag, distance and routes



Waiting at the warehouse is costly: late rate rises from 7.8% for same-day dispatch to 69.7% at two days. Longer distances also run later, but the Remote effect is larger than distance alone explains. 9 of the ten worst routes below are Remote or Tier-3, and every Remote route is above the 27.0% average.



4. Hotspots: where factors combine

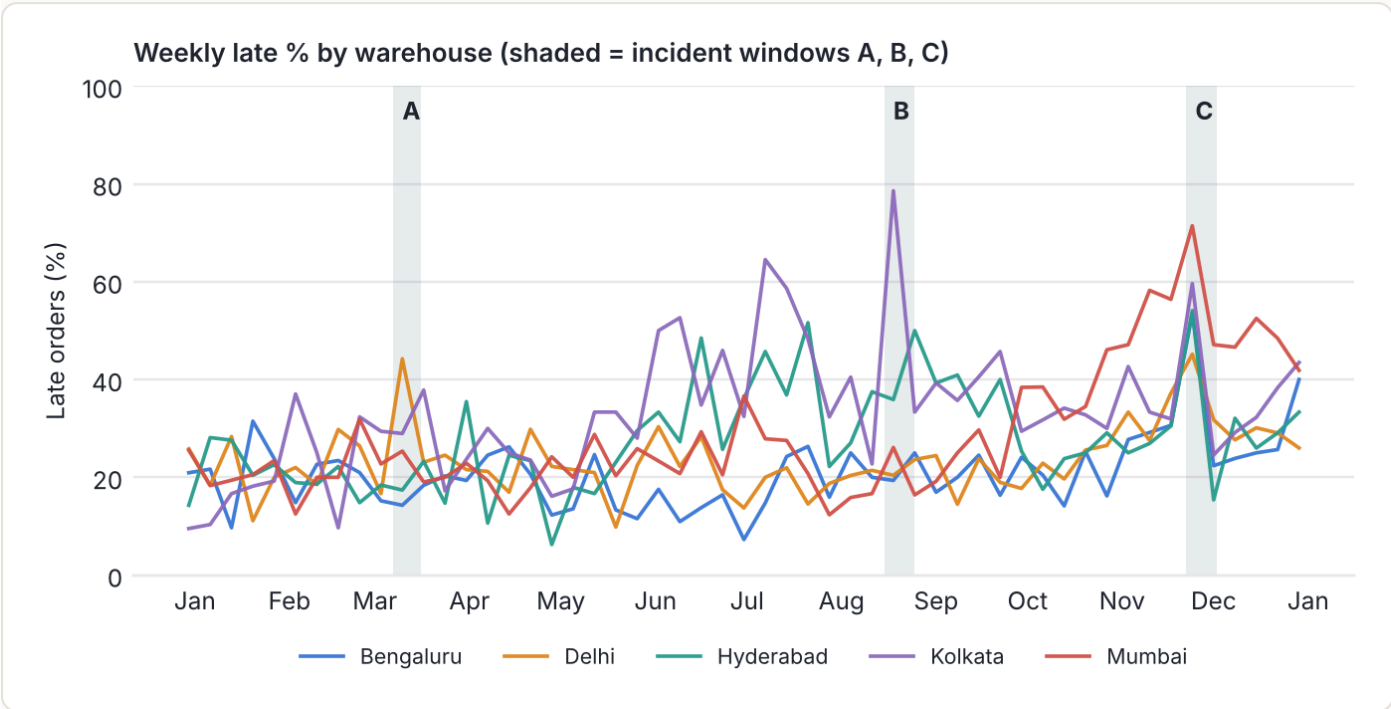


Top 10 warehouse x carrier x category combinations by late % (40+ orders)

Warehouse	Carrier	Category	Orders	Late %	Avg days late
Kolkata	SwiftLine	Appliances	53	66.0%	1.23
Hyderabad	SwiftLine	Furniture	51	64.7%	1.10
Kolkata	SwiftLine	Furniture	44	63.6%	1.52
Mumbai	SwiftLine	Appliances	109	63.3%	1.34
Mumbai	SwiftLine	Furniture	76	59.2%	1.14
Delhi	SwiftLine	Appliances	95	57.9%	1.04
Hyderabad	SwiftLine	Appliances	73	54.8%	1.07
Delhi	SwiftLine	Furniture	83	51.8%	0.84
Mumbai	EcoExpress	Furniture	57	50.9%	0.93
Bengaluru	SwiftLine	Furniture	87	47.1%	0.97

SwiftLine appears in 9 of the 10 worst combinations, and all 10 involve Furniture or Appliances. The worst pockets combine a weak carrier with bulky goods, so both levers (carrier mix and bulky-goods handling) matter together. Mumbai on Fridays (48% late) is the clearest warehouse-day hotspot in the left heatmap.

5. Spikes and incidents

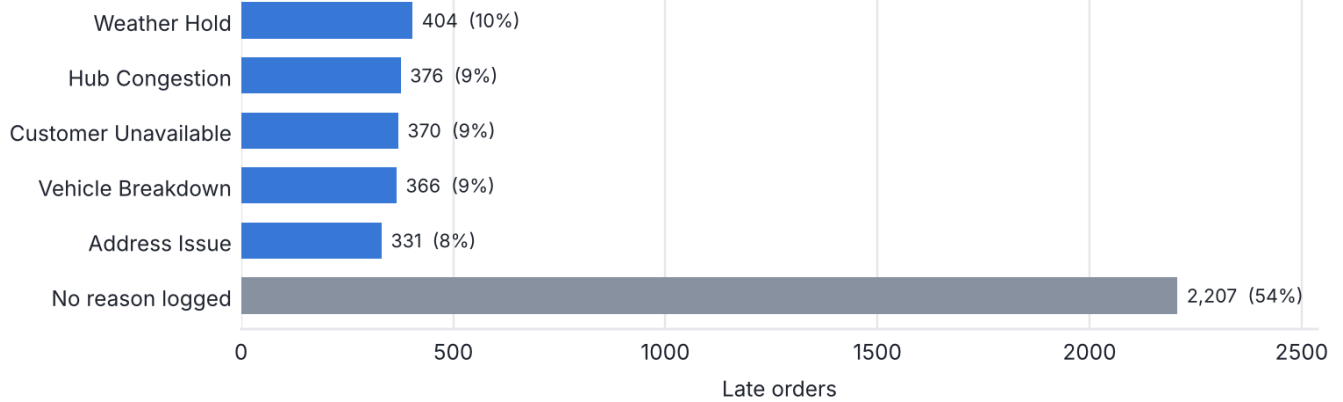


Incident windows

	Incident	Hub	Dates	Orders	Late % in window	Hub baseline	Uplift	Spike-days flagged
A	Transport strike	Delhi Hub	10 Mar – 13 Mar	24	70.8%	24.1%	+47 pts	1
B	Flooding	Kolkata Hub	18 Aug – 22 Aug	28	96.4%	34.2%	+62 pts	0
C	Sale-week surge	All hubs	25 Nov – 29 Nov	285	66.3%	27.0%	+39 pts	9

The spike detector (a warehouse-day with 8+ orders and a late rate at least 2.5 standard deviations above that warehouse's own mean) flagged 15 warehouse-days, 10 of them inside the three planted windows. It catches the sale-week surge (C, 285 orders) clearly. The Kolkata flood (B) is missed by the daily rule because Kolkata sees only 4–7 orders a day, below the 8-order minimum; the weekly chart still shows it, with the week of 18 Aug at 79% late. Windows A and B hold only 24 and 28 orders, so treat them as indicative. For small hubs, monitor weekly rather than daily.

Reason logged in delivery events, late orders only



Only 46% of late orders carry an exception event, so most delays cannot be explained from the event log. Among those that do, weather holds, hub congestion, customer unavailability, vehicle breakdowns and address issues are all of similar size.

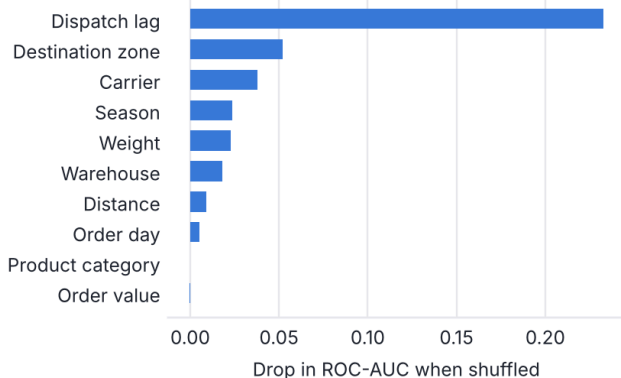
6. Delay-risk model

Three models predict whether an order will be late, using carrier, warehouse, product category, destination zone, order weekday, season, distance, weight, order value and dispatch lag. Training used a 75/25 stratified split (random seed 42); every figure below is on the 3,750 held-out orders.

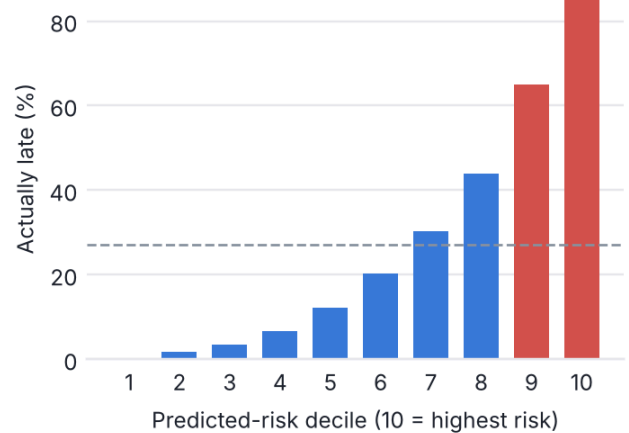
Model	Accuracy	ROC-AUC
Gradient Boosting ★	0.833	0.884
Logistic Regression	0.825	0.875
Decision Tree (depth 4)	0.809	0.829

★ best by ROC-AUC. Metrics on held-out orders.

What drives predicted delay risk



Held-out orders: late rate by risk decile



Gradient Boosting is best (ROC-AUC 0.884). At a 0.5 threshold its precision is 73% and recall 60%. The top three drivers by permutation importance are **Dispatch lag**, **Destination zone** and **Carrier**. The riskiest decile is late 86% of the time versus 0.3% for the safest, a 3.2× lift over the 27% average. The shallow decision tree uses dispatch lag first, then Remote destination, then carrier, which matches the segment analysis.

Caveat: dispatch lag is only known once the order has left the warehouse, so the score is a *post-dispatch* risk score. Drop that feature for a pure at-order-time prediction.

7. Sizing the opportunity

Each row estimates the late orders avoided if a segment performed like a realistic benchmark. Rows overlap (a late Remote order on SwiftLine appears in two), so the estimates should not be added together.

Area	What-if	Orders	Late % now	Benchmark	Late orders avoided	On-time gain
Dispatch lag	2-day-lag orders dispatched with 1-day-lag performance	2,309	69.7%	27.6%	972	+6.5 pts
SwiftLine	SwiftLine matches the other three carriers	4,527	37.1%	22.7%	654	+4.4 pts
Festive season	Oct–Dec matches Jan–Sep	5,227	33.5%	23.6%	519	+3.5 pts
Furniture & Appliances	Bulky goods match all other categories	2,739	40.5%	24.0%	451	+3.0 pts
Mumbai Fridays & festive	Mumbai peak slots match Mumbai's other days	1,653	45.4%	19.2%	433	+2.9 pts
Remote zone	Remote matches Tier-3 performance	1,501	57.5%	33.9%	355	+2.4 pts
Kolkata & Hyderabad monsoon	Monsoon matches their other months	1,243	40.2%	26.8%	167	+1.1 pts

8. Recommendations

- Set a 24-hour dispatch SLA.** Dispatch lag is the strongest single driver: orders dispatched the same day are late 7.8% of the time, versus 27.6% after one day and 69.7% after two. 15% of orders wait two days. Bringing 2-day orders to the 1-day rate would avoid about 972 late orders.
- Rebalance away from SwiftLine.** SwiftLine is late 37.1% of the time against 22.7% for the other carriers. On Remote routes it is late 71.5% (463 orders). Move Remote and bulky volume to BlueArrow (20.2% late) and add SLA penalties plus a monthly carrier scorecard.
- Treat Remote destinations differently.** Remote orders are late 57.5% versus 20.4% for Metro. Quote longer promise dates, or use a regional carrier. Matching Tier-3 performance would avoid about 355 late orders.
- Pre-book peak and weather capacity.** On-time delivery fell to 59.7% in November (it was 78.7% in Jan–May). Mumbai is the hotspot: late 47.7% on Fridays vs 27.8% on other days, and 47.3% in Oct–Dec vs 22.2% earlier. Add Friday and festive shifts at Mumbai, and weather buffers at Kolkata and Hyderabad in monsoon (40.2% late).
- Give bulky goods their own handling.** Furniture and Appliances are late 40.5% of the time against 24.0% for everything else, and 44.5% when the parcel is over 25 kg. Use freight partners and longer promises for heavy bulky items.
- Close the reason-code gap.** 54% of late orders (2,207) have no exception event, so their cause cannot be read from the data. Require a reason code whenever a shipment misses a milestone; that turns this analysis from correlation into diagnosis.
- Use the risk score at dispatch.** The model ranks held-out orders well: the riskiest decile is late 86% of the time against 27% overall (ROC-AUC 0.88). Flag high-risk orders for expedited carriers, proactive customer messages and re-quoted dates.

Method

Orders, shipments, warehouses, carriers and delivery events were merged into one order-level table (one row per order). Delay days = actual delivery date – promised delivery date; an order is **late** if delay days > 0, **on time** otherwise, and **severely late** at 3+ days. Segment tables show late % and average days late per group. Hotspots rank warehouse × carrier × category combinations with at least 40 orders. Spikes use a per-warehouse daily z-score. The exception reason is the first non-routine delivery event per shipment. Season: Winter Jan–Feb, Summer Mar–May, Monsoon Jun–Sep, Festive Oct–Dec.

step / check	rows	detail
Load orders	15,000	one row per order
Load shipments	15,000	one shipment per order
Load warehouses / carriers	9	5 warehouses, 4 carriers
Load delivery events	76,847	5 standard events per shipment + exception events
Orphan shipments (no matching order)	0	join orders ↔ shipments on order_id
Duplicate order IDs	0	none found
Delivered before dispatch	0	date logic check
Promised on/before order date	0	date logic check
Missing values in merged table	0	all key fields complete
Shipments with a logged exception	1,847	first exception event attached; others labelled 'None'
Final analysis table	15,000	29 columns after derived fields

Limitations

Synthetic data: effects were planted by design, so findings illustrate the method rather than any real network. Segment effects overlap, so what-if estimates are indicative. Small incident windows (A, B) have few orders. The model is trained and tested on one year of data; real use needs a time-based split and monitoring for drift.

Reproduce

```
pip install -r requirements.txt
python generate_data.py      # data/*.csv
streamlit run app.py         # interactive dashboard
python build_report.py       # this report
```